



FOR JAVA DEVELOPERS MOVING TO NESTJS

Wrong user. Blocked.

Spring Boot to NestJS, episode 21: rate limiting and throttling

The limit number is the easy part. A rate limiter is a choice of identity, trust boundary and shared state.

Behind a proxy with default settings, client two's first request was refused on both stacks: every request carried the proxy's address.

Trusting the whole X-Forwarded-For header let a client choose its own key: six requests, six 200s. Trusting the one hop you control stopped it after three.

In memory, two instances allowed six of three and a restart reset the count. Redis allowed three and kept it.

In scope

- @nestjs/throttler and Bucket4j in a servlet filter
- Proxy trust: Express trust proxy, Spring forward-headers-strategy
- Keying by a verified user
- Shared storage in Redis
- Window edge behaviour

Not in scope

- Gateway and CDN limits
- Sliding-window algorithms in depth

Spring Boot has no built-in throttler here; Bucket4j in a filter is one common way to build it.

01 The measurements

BEHIND A PROXY, DEFAULT, CLIENT TWO FIRST
REQUEST

429 on both

KEY FOR "attacker, 203.0.113.7"

one hop / native: 203.0.113.7; trust all /
framework: attacker

CLIENT WRITES ITS OWN HEADER, SIX
REQUESTS

trust all: six 200s; one hop: three, then 429

SAME ADDRESS, BOB AFTER ALICE x3

by address 429; by user 200

TWO INSTANCES, SIX ALTERNATING

in memory 6 allowed; Redis 3

RESTART AFTER 200 200 200 429

in memory next 200; Redis next 429

2.2 s REQUEST, THREE PER 2 s

Spring 200; Nest 429 (blockDuration defaults to
ttl)

Three a minute; the edge probe three per two seconds.

02 The mapping

Nest: ThrottlerModule sets the budget, ThrottlerGuard as APP_GUARD enforces it, keyed by req.ip unless you override getTracker. Spring: a Bucket4j bucket per key in a OncePerRequestFilter.

app.ts

```
imports: [ThrottlerModule.forRoot({ throttlers: [{ ttl: 60000, limit: 3 }] })],
providers: [{ provide: APP_GUARD, useClass: ThrottlerGuard }],
```

RateLimitFilter.java

```
Bucket bucket = bucketFor.apply(scope + identity(request));
ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);
if (probe.isConsumed()) { chain.doFilter(request, response); return; }
response.setHeader("Retry-After", String.valueOf(seconds));
response.setStatus(429);
```

five requests at three a minute: 200 200 200 429 429 on both, Retry-After 60

Run on a real machine, 2026-09-30, by scripts/verify-throttle.sh in the course repository.

03 Share the count

A ThrottlerStorage backed by Redis, so every instance counts the same hits. Bucket4j has a Lettuce proxy manager for the same job.

redis-storage.ts

```
async increment(key: string, ttl: number, limit: number) {
  const replies = await this.redis.multi().incr(key).pexpire(key, ttl, 'NX').pttl(key).exec();
  const totalHits = Number(replies![0][1]);
  const seconds = Math.ceil(Number(replies![2][1]) / 1000);
  const isBlocked = totalHits > limit;
  return { totalHits, timeToExpire: seconds, isBlocked, timeToBlockExpire: isBlocked ? seconds : 0 };
}
```

two instances, one Redis, six requests: three allowed; the count survived a restart

Run on a real machine, 2026-09-30, by scripts/verify-throttle.sh in the course repository.

Before you trust a rate limit

1. What is the key: the address, or a verified user?
2. Behind a proxy, trust only the path you control
3. Strip client-supplied forwarded headers at the edge
4. More than one instance? Share the count
5. Test the window edge, not just the count
6. Broad abuse limits often belong at the gateway

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-throttle.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS rate limiting https://docs.nestjs.com/security/rate-limiting

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. Redis is a trademark of Redis Ltd. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.